

Дајкстрин алгоритам

Милан Јанковић
Број индекса: 224/2025
Математички факултет
Универзитет у Београду

29. август 2026.

Садржај

1 Тежински графови	1
2 Дајкстрин алгоритам	2
2.1 Основна идеја	2
2.2 Псеудокод	3
2.3 Сложеност алгоритма	3

Увод

Графови се користе за представљање мноштва различитих система: мрежа путева, комуникационих мрежа, друштвених мрежа, електричних кола и других система у којима постоје објекти (чворови) и везе међу њима (гране). Ако се свакој грани придружи неки број који представља њену цену, дужину или време преласка, добија се *тежински граф*.

Један од најпознатијих проблема везаних за тежинске графове јесте налажење најкраћег пута од једног чвора до осталих чворова у графу. Овим проблемом, као и његовим ефикасним решењем, бави се овај рад.

1 Тежински графови

Дефиниција 1.1. Тежински граф $G = (V, E, w)$ састоји се од скупа чворова V , скупа грана E и функције тежине $w : E \rightarrow \mathbb{R}$, која свакој грани $(u, v) \in E$ додељује њену тежину $w(u, v)$.

У овом раду ће се претпостављати да су све тежине грана ненегативне, односно да важи $w(u, v) \geq 0$ за сваку грану $(u, v) \in E$, пошто је то услов који Дајкстрин алгоритам захтева.

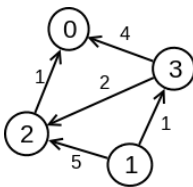
Граф се у рачунару најчешће представља матрицом суседства или листом суседства. Код матрице суседства, поље (i, j) садржи тежину гране од

чвора i до чвора j , док листа суседства сваком чвору придружује листу парова (сусед, тежина гране до тог суседа). Поређење ове две репрезентације дато је у табели 1.

Особина	Матрица суседства	Листа суседства
Меморија	$O(n^2)$	$O(n + m)$
Провера гране (i, j)	$O(1)$	$O(\deg(i))$
Погодно за	густе графове	ретке графове

Табела 1: Поређење репрезентација графа

Пример 1.1. На слици 1 приказан је један мали тежински.



Слика 1: Пример тежинског графа

2 Дајкстрин алгоритам

2.1 Основна идеја

Дајкстрин алгоритам одређује најкраћи пут од једног, унапред изабраног почетног чвора до свих осталих чворова у тежинском графу без грана негативне тежине.

Дефиниција 2.1. Нека је $d(v, u)$ дужина најкраћег пута од почетног чвора v до чвора u . *Релаксација* гране (u, z) представља проверу да ли важи

$$d(v, u) + w(u, z) < d(v, z),$$

и уколико важи, ажурирање тренутне процене $d(v, z)$ на мању вредност $d(v, u) + w(u, z)$.

Алгоритам гради решење постепено. На почетку је растојање до почетног чвора једнако 0, а до свих осталих чворова једнако $+\infty$. У сваком следећем кораку бира се необрађени чвор са тренутно најмањом проценом растојања – за тај чвор се показује да му је та процена уједно и коначна, најкраћа могућа вредност. Након тога се тај чвор проглашава обрађеним, а сви његови суседи се релаксирају. Поступак се понавља док се не обраде сви чворови.

Теорема 2.1. *Ако граф G нема грана негативне тежине, Дајкстрин алгоритам исправно одређује најкраћи пут од почетног чвора v до сваког другог чвора u у G .*

Напомена 2.1. *Уколико граф садржи гране негативне тежине, Дајкстрин алгоритам може дати погрешан резултат. У том случају се уместо њега користи, на пример, Белман–Фордов алгоритам.*

2.2 Псеудокод

Извршавање алгоритма може се описати следећим корацима:

1. Растојање до почетног чвора поставити на 0, а до свих осталих чворова на $+\infty$.
2. Све чворове сместити у ред са приоритетом, уређен према тренутној процени растојања.
3. Док год ред није празан, извући чвор u са најмањом тренутном проценом растојања и прогласити га обрађеним.
4. За сваког суседа z чвора u извршити релаксацију гране (u, z) .
5. Вратити се на претходни корак све док се не обраде сви чворови.

2.3 Сложеност алгоритма

Уколико се за имплементацију користи листа суседства и ред са приоритетом (хип), временска сложеност алгоритма износи

$$O((n + m) \log n),$$

где је n број чворова, а m број грана графа. Захваљујући оваквој сложености, Дајкстрин алгоритам је применљив и на графове са великим бројем чворова и грана.

Закључак

Дајкстрин алгоритам представља ефикасно решење проблема налажења најкраћих путева од једног чвора до свих осталих чворова у тежинском графу без грана негативне тежине. Захваљујући коришћењу реда са приоритетом, алгоритам постиже сложеност $O((n + m) \log n)$, због чега представља једну од основних техника које се изучавају у оквиру алгоритама на графовима.